

Embodied.cpp: A Portable Inference Runtime of Embodied AI Models on Heterogeneous Robots

Ling Xu, Chuyu Han, Borui Li, Hao Wu, Shiqi Jiang, Ting Cao, Chuanyou Li, Sheng Zhong, Shuai Wang

ABSTRACT

Embodied AI models now span vision-language-action (VLA) models and world-action models (WAMs), but practical deployment remains fragmented across model-specific Python stacks, backend assumptions, and robot-side glue code, especially on heterogeneous edge devices. Existing inference runtimes are designed mainly for request-response serving and therefore do not satisfy the runtime contract of embodied deployment: multi-rate execution inside closed-loop control, latency-first batch-1 inference on heterogeneous hardware, and extensible embodied interfaces beyond fixed token I/O. We present Embodied.cpp, a portable C++ inference runtime for embodied models. Based on an architectural analysis of representative VLA models and WAMs, Embodied.cpp captures a shared execution path and organizes it into five layers: input adapters, sequence builders, backbone execution, head plugins, and deployment adapters. The runtime provides modular multi-rate execution, latency-first fused inference, and extensible operator and I/O support, enabling deployment across heterogeneous devices, robots, and simulators through one backend abstraction. We evaluate Embodied.cpp on two VLA models, HY-VLA and pi0.5, and on a preliminary WAM benchmark using a LingBot-VA Transformer block. The VLA deployments achieve successful closed-loop execution with 100.0% and 91.0% task success rates, respectively. The WAM benchmark reduces block memory from 312.2 MiB to 88.1 MiB. These results show that Embodied.cpp improves deployment efficiency while preserving high accuracy across diverse embodied model architectures.

About this document

This is a **Reviewed Preprint**: a third-party paper that llmXive has *auto-reviewed* (with its LLM review panel) but *never modified*. The original manuscript follows this cover page **exactly as published** by its authors — llmXive re-typesets nothing and claims no authorship.

Provenance. Ingested by llmXive on 2026-07-07 — scraped from arXiv; via llmXive dashboard, GitHub issue #520; submitted by github-actions[bot].

Source. <https://arxiv.org/abs/2607.02501>

About llmXive. llmXive is an automated platform for open scientific discovery. Its specialist LLM agents advance their own ideas from brainstorm to peer-reviewed paper, and they also review notable third-party papers — drawn from the most-downloaded papers on Hugging Face and from submissions by human contributors. Every submitted paper is reviewed by the LLM panel *and* used to seed a new llmXive follow-up study. This paper is one such third-party work: llmXive reviewed it but did not write it. Learn more at <https://github.com/ContextLab/llmXive>.

Automated review. See the accompanying [peer-review-llmxive.pdf](#) for the full reviewer report.

llmXive follow-up. A separate llmXive study building on this work: [PROJ-998-llmxive-follow-up-extending-embodied-cpp](#).

Embodied.cpp: A Portable Inference Runtime of Embodied AI Models on Heterogeneous Robots

Ling Xu¹ Chuyu Han² Borui Li^{1,†} Hao Wu^{2,†} Shiqi Jiang³ Ting Cao⁴
Chuanyou Li¹ Sheng Zhong² Shuai Wang¹

¹  Southeast University ²  Nanjing University ³  Microsoft Research

⁴  Institute for AI Industry Research (AIR), Tsinghua University

[†]Project Leader

Embodied AI models now span vision-language-action (VLA) models and world-action models (WAMs), but practical deployment remains fragmented across model-specific Python stacks, backend assumptions, and robot-side glue code, especially on heterogeneous edge devices. Existing inference runtimes are designed mainly for request-response serving and therefore do not satisfy the runtime contract of embodied deployment: multi-rate execution inside closed-loop control, latency-first batch-1 inference on heterogeneous hardware, and extensible embodied interfaces beyond fixed token I/O. We present Embodied.cpp, a portable C++ inference runtime for embodied models. Based on an architectural analysis of representative VLA models and WAMs, Embodied.cpp captures a shared execution path and organizes it into five layers: input adapters, sequence builders, backbone execution, head plugins, and deployment adapters. The runtime provides modular multi-rate execution, latency-first fused inference, and extensible operator and I/O support, enabling deployment across heterogeneous devices, robots, and simulators through one backend abstraction. We evaluate Embodied.cpp on two VLA models, HY-VLA and pi0.5, and on a preliminary WAM benchmark using a LingBot-VA Transformer block. The VLA deployments achieve successful closed-loop execution with 100.0% and 91.0% task success rates, respectively. The WAM benchmark reduces block memory from 312.2 MiB to 88.1 MiB. These results show that Embodied.cpp improves deployment efficiency while preserving high accuracy across diverse embodied model architectures.

Project Link: <https://github.com/SEU-PAISys/Embodied.cpp>

1. Introduction

Academia and industry have already produced a rapidly growing set of embodied models. Recent systems now span vision-language-action (VLA) models and world-action models (WAMs), showing substantial progress in model architecture and capability [1, 2, 3, 4, 5, 6, 7]. At the same time, the surrounding ecosystem for data, training, and simulation has also improved substantially through projects such as LeRobot, Open X-Embodiment, ManiSkill, LIBERO, and Isaac Sim [8, 9, 10, 11, 12]. But practical impact does not end at model construction. To become reliable robot-side systems, these models must run on heterogeneous and resource-constrained devices, from Jetson and RK-based boards to x86 edge boxes and workstation-class robots, without rebuilding the software stack for each new model family.

Achieving this requires a unified inference runtime. Existing inference runtimes, however, do not yet provide such support. General LLM or VLM runtimes are designed for request-response serving, relatively

uniform token interfaces, and throughput-oriented optimization [13, 14, 15, 16]. Embodied inference, by contrast, lives inside a closed-loop interaction process with robot- and simulator-side dependencies. As a result, even a strong checkpoint still has to be stitched into Python research code, backend-specific inference paths, handwritten sensor wrappers, and platform-specific control logic before it can act on a robot. As embodied architectures diversify, this integration burden only grows, making a portable inference runtime for embodied AI models increasingly necessary.

The challenge arises because embodied deployment fundamentally changes the runtime contract. Compared with conventional LLM or VLM serving, an embodied inference runtime must satisfy three key requirements. **(1) Multi-rate execution.** Embodied inference is not a single synchronous forward pass: perception encoders, transformer backbones, predictive branches, and action heads may need to run at different rates inside the same control loop, as increasingly visible in hierarchical and fast-slow VLA systems as well as recent WAMs [3, 4, 5]. **(2) Latency-first closed-loop control.** The optimization target is not throughput but stable closed-loop control, which requires low latency, low jitter, and efficient batch-1 execution on heterogeneous edge hardware. **(3) Extensible embodied interfaces.** Embodied deployment must go beyond fixed token I/O and absorb custom operators, embodied multimodal inputs, and heterogeneous outputs ranging from action chunks to predicted futures. These are not peripheral engineering details; together they define the runtime contract that a practical embodied inference runtime must satisfy.

Motivated by these observations, we present `Embodied.cpp`, a portable C++ inference runtime for embodied models. Its design directly addresses the three challenges: modular multi-rate execution decouples components with different refresh frequencies; latency-first fused execution targets predictable small-batch control on heterogeneous devices; and extensible operator and embodied I/O support turns model-specific heads, interfaces, and backends into pluggable runtime modules instead of one-off glue code. These principles are realized in a unified five-layer architecture spanning input adapters, sequence builders, backbone execution, head plugins, and deployment adapters. The result is an inference runtime with a stable reusable core, while still leaving enough extension surface for new embodied model variants.

We make three contributions:

- An architectural analysis of representative embodied models spanning vision-language-action (VLA) models and world-action models (WAMs), revealing a shared execution path whose main divergences are confined to a small set of pluggable heads and predictive modules.
- The design of `Embodied.cpp` as a portable C++ inference runtime that maps this shared path into five layers: input adapters, sequence builders, backbone execution, head plugins, and deployment adapters, with backend abstraction for heterogeneous edge devices.
- We evaluate `Embodied.cpp` on two widely used VLA models and one representative WAM benchmark, showing improved efficiency while maintaining high accuracy.

2. Related Work and Motivation

2.1. Embodied AI Model and Architectural Analysis

From an inference-runtime perspective, recent embodied models can be organized into two online families: vision-language-action (VLA) models and world-action models (WAMs). As summarized in Figure 1 and Table 1, VLA models primarily realize a perception-to-action path, while WAMs make future

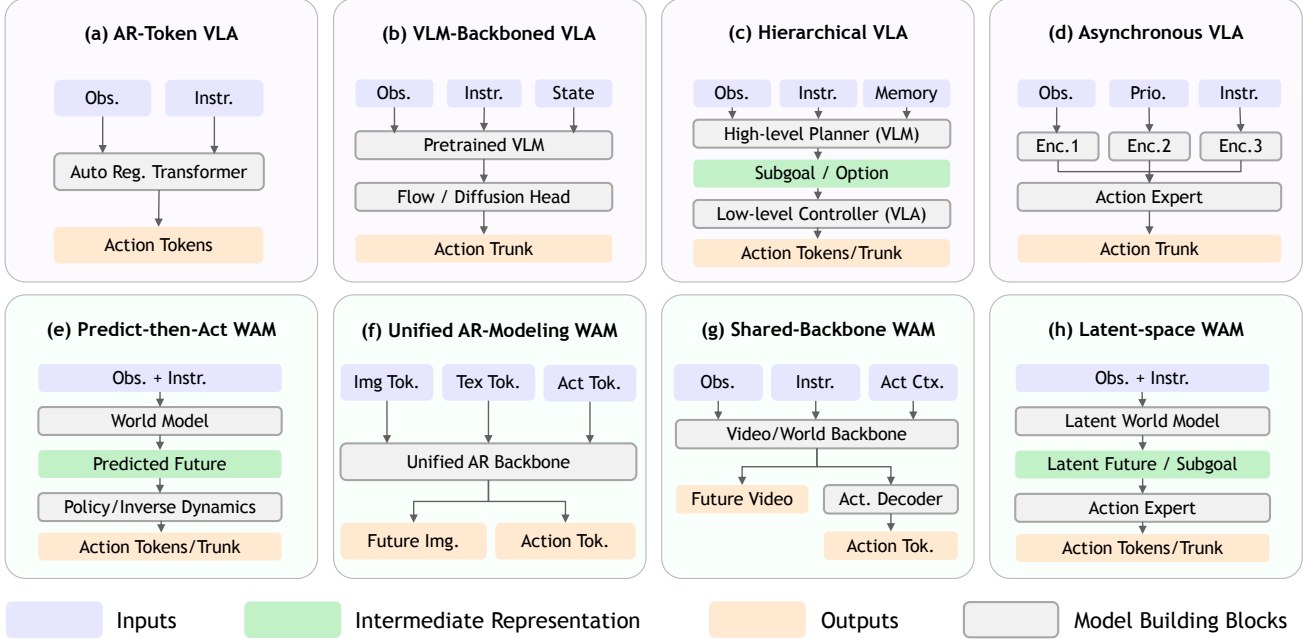


Figure 1 | Architectural taxonomy of embodied models. We first separate VLA models (a-d) from WAMs (e-h), and then organize each family by internal inference structure.

prediction an explicit part of online control [1, 2, 6]. This split matters directly to systems design because it determines whether the runtime mainly executes an action policy or must jointly maintain predictive state and action generation.

Within the VLA family, the architectural progression is from monolithic action generation to increasingly structured modular execution. AR-Token VLA models such as RT-2 and OpenVLA use one shared backbone to autoregressively generate action tokens [17, 1]. VLM-Backboned VLA models such as Octo, pi0, pi0.5, and MuseVLA still rely on a strong shared visual-language backbone, but pair it with more explicit continuous-action heads [18, 2, 3, 19]. More recent systems then branch into two modular directions. Hierarchical VLA models split semantic planning from low-level control, with a planner producing subgoals for a downstream controller, as in Hi Robot, GeneralVLA, RT-H, and Gemini Robotics 1.5 [20, 21, 22, 23]. Asynchronous VLA models instead split execution by time scale: decoupled modules run at different rates and coordinate through buffered state, as in GR00T N1, Fast-in-Slow, and DAM-VLA [4, 24, 25]. The key runtime takeaway is that even within VLA models, the execution unit is no longer uniformly a single synchronous forward pass.

The WAM family is organized by how future prediction is coupled with action generation. Predict-then-Act WAMs such as UniPi keep the two stages explicit: a world model predicts future states, and a downstream action expert consumes them [26]. Unified AR-Modeling WAMs collapse future modeling and action generation into one autoregressive token space, as in WorldVLA and LingBot-VA [27, 5]. Shared-Backbone WAMs reuse one backbone across world modeling and action generation, while still exposing separate blocks that can potentially be scheduled or extended independently, as in DreamZero, Fast-WAM, Cosmos Policy, and the Unified Video Action Model (UWM) [28, 29, 30, 31]. Latent-space WAMs compress the predictive path further by generating a compact latent future or subgoal for a downstream action expert, as in LaWAM and Being-H0.7 [32, 33]. Compared with VLA models, WAMs

Table 1 | Architecture-oriented comparison of embodied AI models.

Subtype	Architectural Characteristic	Structure	Representative Models
Vision-Language-Action (VLA) Models			
AR-Token VLA	One backbone autoregressively generates action tokens.	Monolithic	OpenVLA [1], RT-2 [17]
VLM-Backboned VLA	A shared pretrained VLM feeds a continuous action head.	Modular	Octo [18], pi0 [2], pi0.5 [3], MuseVLA [19]
Hierarchical VLA	A high-level planner produces subgoals for a low-level controller.	Modular	Hi Robot [20], GeneralVLA [21], RT-H [22], Gemini Robotics 1.5 [23]
Asynchronous VLA	Decoupled modules run at different rates through buffered coordination.	Modular	GR00T N1 [4], Fast-in-Slow [24], DAM-VLA [25]
World-Action Models (WAM)			
Predict-then-Act WAM	An explicit world model predicts future states before a downstream action expert acts.	Modular	UniPi [26]
Unified AR-Modeling WAM	Future world and robotic action are jointly generated in one autoregressive token space.	Monolithic	WorldVLA [27], LingBot-VA [5]
Shared-Backbone WAM	World modeling and action generation share one backbone, while auxiliary blocks can run at different rates.	Modular	DreamZero [28], Fast-WAM [29], Cosmos Policy [30], UWM [31]
Latent-space WAM	A compact latent future or subgoal is predicted and then consumed by an action expert.	Modular	LaWAM [32], Being-H0.7 [33]

therefore broaden the runtime contract from direct action decoding to coordinated management of predictive computation and control computation.

Taken together, Figure 1 and Table 1 suggest three implications for inference runtime design. First, structural organization is now a first-class model property: some systems remain monolithic, but many recent ones are better understood as interacting planner, backbone, world-model, and action modules. Second, intermediate states such as subgoals, buffered context, predicted futures, and latent futures have become explicit runtime objects rather than hidden internal details. Third, timing structure is increasingly model-defined, especially in hierarchical and asynchronous systems, so a practical embodied inference runtime must support stateful multi-component orchestration instead of assuming one uniform synchronous request-response path.

2.2. Existing Inference Runtimes for AI Models

AI model inference is well-studied in the past few years [34, 35, 36, 37, 38, 39]. General-purpose inference runtimes already cover much of the shared execution substrate. llama.cpp emphasizes lightweight C/C++ local inference, broad hardware coverage, and practical packaging [13]; ONNX Runtime provides cross-platform acceleration through multiple execution providers [14]; and serving systems such as SGLang and vLLM-Omni target large-scale LLM/VLM inference with richer multimodal pipelines [15, 16]. These systems are strong inference runtimes, but they primarily target request-response workloads with relatively uniform interfaces.

Embodied settings impose a different contract: closed-loop control, heterogeneous inputs and outputs,

Table 2 | Comparison of inference runtimes from the perspective of embodied model coverage and deployment readiness. ‘VLA’ and ‘WAM’ denote first-class support for the corresponding embodied model families. ‘Modular’ denotes support or optimization for multi-component embodied models rather than only monolithic inference graphs. ‘Edge’ denotes whether the runtime can run with deployment-oriented optimization on edge or end-side devices. ‘Hetero. HW’ denotes whether one runtime can jointly use heterogeneous devices rather than only switch among backends. ✓: native support. ✗: unsupported. ◦: achievable only with substantial custom integration. △: limited or partial support.

System	VLA	WAM	Modular	Edge	Hetero. HW	Robot	Simulator
llama.cpp [13]	✗	✗	✗	✓	△	✗	✗
ONNX Runtime [14]	◦	◦	◦	✓	✓	✗	✗
SGLANG [15]	✗	✗	✗	✗	✗	✗	✗
vLLM-OMNI [16]	◦	◦	△	✗	△	✗	✗
vla.cpp [40]	✓	✗	△	✓	△	✓	△
Embodied.cpp (ours)	✓	✓	✓	✓	✓	✓	✓

persistent state, batch-1 latency sensitivity, and direct robot or simulator integration. Simulation platforms such as ManiSkill, LIBERO, and Isaac Sim are important for training and benchmarking [10, 11, 12], but they are not themselves embodied inference runtimes. The closest recent step is vla.cpp, which brings seven VLA architectures into one portable C++ inference runtime and reports execution from consumer GPUs to embedded devices together with an on-robot stress test [40]. This is an important milestone, but it remains VLA-centric.

Table 2 makes the gap concrete from two complementary angles: model-family coverage and deployment-facing capability. The first question is whether a runtime directly supports VLA models or WAMs, and whether it provides explicit optimization support for modular multi-component models rather than only monolithic inference graphs. The second is whether it can actually carry those models to embodied deployment, including edge execution, heterogeneous-device cooperation, and direct connection to robots or simulators. Existing systems are either generic inference runtimes without first-class embodied model support, or embodied inference runtimes specialized to only one model family.

Overall, existing systems still stop short of first-class embodied model coverage, modular-model optimization, and direct deployment interfaces. What remains underbuilt is an embodied inference runtime that jointly supports both VLA and WAM execution while also landing on edge devices, heterogeneous hardware, robots, and simulators.

3. Project Overview

3.1. Challenges

Embodied deployment should first be understood by contrast with traditional LLM and VLM inference. Conventional language or multimodal serving usually assumes a mostly synchronous request-response path, a relatively uniform token interface, and throughput-oriented optimization over large batches or many concurrent users. Embodied.cpp faces a different setting: embodied models run inside closed-loop control, combine multiple heterogeneous modules, and must remain deployable on robot-side hardware while absorbing new model families over time. For an open-source embodied inference runtime project,

these differences collapse into three practical system challenges, each of which directly shapes the design of `Embodied.cpp`.

- **1. Multi-rate execution.** As an open-source project, `Embodied.cpp` must support not only today’s embodied models, but also the optimizations that future embodied architectures may require. This is difficult because many modern embodied systems are no longer monolithic. VLA models and WAMs are increasingly assembled from multiple modules, such as perception encoders, transformer backbones, predictive branches, and action heads. Unlike a conventional LLM runtime, efficient embodied inference does not always require every module to run at every step. A perception stack may refresh less frequently, a predictive branch may run only when future estimation is needed, and an action head may need to execute at a much higher control rate.
- **2. Latency-first closed-loop control.** Embodied deployment also raises a harder performance problem than ordinary cloud inference. In many robotics settings, execution is effectively batch-1: a single robot or simulator must receive actions continuously, with low latency, low jitter, and predictable timing behavior. At the same time, deployment targets are heterogeneous, spanning Jetson devices, RK-based platforms, x86 edge boxes, and workstation-class systems. This creates a tension between latency-first execution and the fused-inference techniques needed to make small-batch execution efficient across different backends and devices.
- **3. Extensible embodied interfaces.** Finally, implementing an embodied inference runtime is not only a scheduling problem. New model families regularly introduce new dependency stacks, larger custom operators, and new input/output conventions. A practical open-source inference runtime must therefore absorb model-specific libraries, cover a broader operator surface, and support new embodied data types on both sides of the interface. Inputs may include images, language, proprioception, history, force or tactile signals, and simulator-provided state. Outputs may be discrete action tokens, continuous action vectors, action chunks, world predictions, or intermediate control representations.

Taken together, these challenges explain why embodied deployment should not be framed as a small extension of an LLM runtime. The reusable substrate remains real and important, especially around multimodal projection, transformer-style execution, backend portability, and operator reuse. But the top-level contract has shifted from token serving to embodied control, and an open runtime must remain extensible enough to absorb future embodied optimizations rather than hard-coding today’s model structure.

3.2. Design Principles and Runtime Architecture

These challenges, in turn, motivate three design principles of `Embodied.cpp`:

- **1. Modular multi-rate execution.** `Embodied.cpp` should expose explicit execution units, pluggable modules, shared state or feature pools, and configurable refresh policies so that different components can run at different rates without forcing a single synchronous path.
- **2. Latency-first fused execution.** `Embodied.cpp` should prioritize stable control performance while still supporting graph replay, buffer reuse, operator fusion, backend-specific dispatch, and careful host-device data movement for efficient small-batch inference on heterogeneous devices.
- **3. Extensible operator and I/O support.** `Embodied.cpp` should provide typed embodied interfaces, pluggable heads, first-class deployment adapters, and enough backend and operator

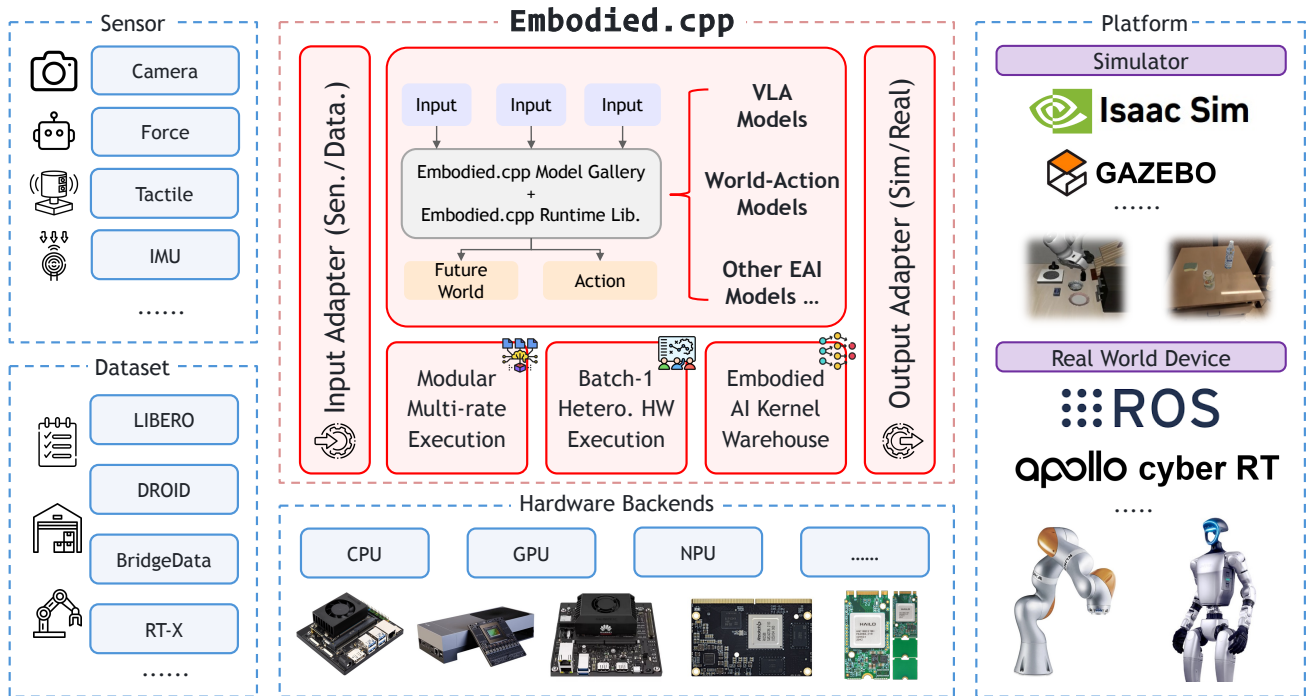


Figure 2 | Project overview of Embodied.cpp. Diverse sensors and datasets enter through input adapters, model execution is unified around one embodied-model runtime that covers VLA models, WAMs, and future variants, and outputs are bridged to simulators and real robots through deployment adapters. Beneath this shared path, three runtime capabilities directly address the core challenges: modular multi-rate execution, latency-first batch-1 execution on heterogeneous hardware, and an embodied AI kernel warehouse for reusable operators and model-specific kernels.

coverage to make new embodied paradigms implementable without rebuilding the surrounding runtime each time.

The model-side analysis in Section 2 and the runtime gap in Table 2 together motivate a runtime architecture that stays uniform at the deployment boundary while remaining extensible inside. Recent systems suggest three complementary lessons: *vla.cpp* shows that a broad family of VLA models can be organized around one shared inference path and one portable bundle format [40]; *Pie* shows the value of separating a fixed execution substrate from a programmable control layer [41]; and *FlashRT* shows that latency-first execution benefits from treating execution state as a first-class runtime object rather than as an implicit cache [42]. Embodied.cpp combines these lessons in a form specialized for embodied deployment.

Figure 2 summarizes this design at a high level. On the left, *input adapters* absorb both online sensor streams and offline dataset samples, so cameras, force or tactile signals, IMU data, and benchmark datasets enter the runtime through one typed embodied interface. In the center, Embodied.cpp maintains one shared embodied-model execution zone that can host VLA models, WAMs, and future embodied variants, while preserving explicit interfaces for future prediction, action experts, and final action generation. Beneath this model zone, three supporting subsystems directly mirror the challenges in Section 3.1: *modular multi-rate execution* handles decoupled scheduling and runtime state, *latency-*

Table 3 | VLA deployment results.

Deployed Model	Model Backbone	Action Chunk	Success Rate (%)	Step (ms)	Inf. (ms)	VRAM (MiB)
HY-VLA	Hunyuan-VL	20	100.0 [83.9, 100.0]	735.9	1340.3	6850
pi0.5	PaliGemma	50	91.0 [86, 94]	56.85	266.6	6546

Table 4 | LingBot-VA single-block quantization microbenchmark.

Inference Runtime	Model Quantization	Latency / block (ms)	Memory / block (MiB)	MAE ↓	Cosine ↑
Python original	BF16	3.236	312.2	0	1
Embodied.cpp	Q4_K	3.171	88.1	$< 3.3 \times 10^{-2}$	$> 9.997 \times 10^{-1}$

first batch-1 execution on heterogeneous hardware targets low-latency deployment across CPUs, GPUs, NPUs, and other accelerators, and the *embodied AI kernel warehouse* collects reusable operators and model-specific kernels needed by evolving embodied architectures. On the right, *output adapters* bridge runtime outputs to simulators and real-world robot software stacks. In this way, `Embodied.cpp` keeps the deployment boundary stable even as model structure, hardware targets, and embodied interfaces continue to evolve.

4. Evaluation

This revision reports two kinds of evidence: closed-loop results for two VLA models, HY-VLA and pi0.5, and a preliminary WAM microbenchmark on LingBot-VA. Full LingBot-VA closed-loop results are not included because the complete model is not yet stable on the constrained local edge setup used in this draft.

VLA model evaluation. We evaluate HY-VLA and pi0.5 through the C++ deployment path. HY-VLA is tested on the RoboTwin `place_empty_cup` task with its corresponding checkpoint, and pi0.5 is tested with its corresponding C++ deployment configuration. We report success rate, action chunk length, server-side inference latency, amortized environment-step latency, and peak GPU memory.

Table 3 shows that both models run correctly through the C++ runtime while preserving task behavior. HY-VLA reaches a 100.0% success rate on the evaluated RoboTwin task. Its latency is higher than pi0.5 because it uses a larger Hunyuan-VL backbone, three-view inputs, and a video-history/MEM vision path. By contrast, pi0.5 uses a lighter PaliGemma backbone and a longer action chunk, which lowers the amortized step cost. Together, these results show that `Embodied.cpp` can support different VLA architectures within one C++ runtime, while latency and memory still depend on backbone size, visual input complexity, and action chunking.

WAM evaluation. For a preliminary WAM evaluation, we use LingBot-VA. Because the full model is not yet stable on the constrained local edge device, we benchmark only its first Transformer block and compare the original PyTorch implementation with the corresponding `Embodied.cpp` implementation.

Table 4 reports the single-block results. The Python baseline uses the original `WanTransformerBlock`

with BF16 weights, while the C++ path uses the corresponding GGUF Q4_K quantized block. Latency is measured after warmup and includes only backend computation for one Transformer block, excluding request parsing, tensor staging, output post-processing, and other server-side overheads. Memory denotes resident weight memory for the measured block.

Using 100 random input samples, we compare the C++ Q4_K block against the Python BF16 block with mean absolute error (MAE) and cosine similarity. The quantized C++ block reduces resident weight memory from 312.2 MiB to 88.1 MiB while keeping MAE below 3.3×10^{-2} and cosine similarity above 9.997×10^{-1} . This is not a full WAM closed-loop result, but it provides initial evidence that `Embodied.cpp` can host and validate WAM-side Transformer components with large memory savings and very limited output drift.

5. Conclusion

The evidence from recent models points in one direction: embodied deployment is converging on a shared execution path even though model families continue to diversify. `Embodied.cpp` captures this convergence in an inference runtime that treats the common path as infrastructure and the diverging parts as plugins. Its five-layer architecture keeps interaction pattern, I/O semantics, objective, and deployment boundary explicit, while reusing the same backbone execution path across paradigms. In the current revision, we validate the C++ inference path quantitatively on two VLA models and position WAMs through architectural analysis. As new embodied model variants emerge, this separation between a stable core and pluggable task-specific components will only compound in value.

References

- [1] Karl Pertsch et al. OpenVLA: An Open-Source Vision-Language-Action Model. *arXiv preprint arXiv:2406.09246*, 2024. URL: <https://arxiv.org/abs/2406.09246>.
- [2] Kevin Black et al. Pi0: A Vision-Language-Action Flow Model for General Robot Control. *arXiv preprint arXiv:2410.24164*, 2024. URL: <https://arxiv.org/abs/2410.24164>.
- [3] Physical Intelligence et al. Pi0.5: A Vision-Language-Action Model with Open-World Generalization. *arXiv preprint arXiv:2504.16054*, 2025. URL: <https://arxiv.org/abs/2504.16054>.
- [4] Johan Bjorck et al. GR00T N1: An Open Foundation Model for Generalist Humanoid Robots. *arXiv preprint arXiv:2503.14734*, 2025. URL: <https://arxiv.org/abs/2503.14734>.
- [5] L. Li et al. Causal World Modeling for Robot Control. *arXiv preprint arXiv:2601.21998*, 2026. URL: <https://arxiv.org/abs/2601.21998>.
- [6] S. Wang, J. Shi, Z. Fu, X. He, F. Liu, C. Yang, Y. Zhou, Z. Fei, J. Gong, J. Fu, M. Z. Shou, X. Huang, X. Qiu, and Y.-G. Jiang. World Action Models: The Next Frontier in Embodied AI. *arXiv preprint arXiv:2605.12090*, 2026. Submitted May 12, 2026. URL: <https://arxiv.org/abs/2605.12090>.
- [7] Xueying Li, Feng Lyu, Hao Wu, Mingliu Liu, Jia-Nan Liu, and Guozi Liu. Stop wandering: Efficient vision-language navigation via metacognitive reasoning. *arXiv preprint arXiv:2604.02318*, 2026.

-
- [8] Hugging Face. LeRobot. GitHub repository, 2026. Accessed June 17, 2026. URL: <https://github.com/huggingface/lerobot>.
 - [9] Open X-Embodiment Collaboration. Open X-Embodiment. Project website, 2026. Accessed June 17, 2026. URL: <https://robotics-transformer-x.github.io/>.
 - [10] ManiSkill Team. ManiSkill. Project website, 2026. Accessed June 17, 2026. URL: <https://maniskill.ai/>.
 - [11] LIBERO Team. LIBERO. Project website, 2026. Accessed June 17, 2026. URL: <https://libero-project.github.io/>.
 - [12] NVIDIA. Isaac Sim. Product website, 2026. Accessed June 17, 2026. URL: <https://developer.nvidia.com/isaac/sim>.
 - [13] Georgi Gerganov et al. llama.cpp. GitHub repository, 2026. 2023–2026. URL: <https://github.com/ggml-org/llama.cpp>.
 - [14] Microsoft. ONNX Runtime Documentation. Official documentation, 2026. Accessed June 17, 2026. URL: <https://onnxruntime.ai/docs/>.
 - [15] LMSYS Org. SGLang. Official documentation and repository, 2026. Accessed June 17, 2026. URL: <https://docs.sglang.io/>.
 - [16] vLLM Project. vLLM-Omni. Official documentation and repository, 2026. Accessed June 17, 2026. URL: <https://docs.vllm.ai/projects/vllm-omni/en/latest/>.
 - [17] Anthony Brohan et al. RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control. *arXiv preprint arXiv:2307.15818*, 2023. URL: <https://arxiv.org/abs/2307.15818>.
 - [18] Octo Model Team et al. Octo: An Open-Source Generalist Robot Policy. *arXiv preprint arXiv:2405.12213*, 2024. URL: <https://arxiv.org/abs/2405.12213>.
 - [19] MuseVLA: An Adaptive Multimodal Sensing Vision-Language-Action Model for Robotic Manipulation. *arXiv preprint arXiv:2606.17598*, 2026. URL: <https://arxiv.org/abs/2606.17598>.
 - [20] L. X. Shi et al. Hi Robot: Open-Ended Instruction Following with Hierarchical Vision-Language-Action Models. *arXiv preprint arXiv:2502.19417*, 2025. URL: <https://arxiv.org/abs/2502.19417>.
 - [21] GeneralVLA: Generalizable Vision-Language-Action Models with Knowledge-Guided Trajectory Planning. *arXiv preprint arXiv:2602.04315*, 2026. URL: <https://arxiv.org/abs/2602.04315>.
 - [22] Suneel Belkhale et al. RT-H: Action Hierarchies Using Language. *arXiv preprint arXiv:2403.01823*, 2024. URL: <https://arxiv.org/abs/2403.01823>.
 - [23] Google DeepMind et al. Gemini Robotics 1.5: Pushing the Frontier of Generalist Robots with Advanced Embodied Reasoning, Thinking, and Motion Transfer. *arXiv preprint arXiv:2510.03342*, 2025. URL: <https://arxiv.org/abs/2510.03342>.
 - [24] Fast-in-Slow: A Dual-System Foundation Model Unifying Fast Manipulation within Slow Reasoning. *arXiv preprint arXiv:2506.01953*, 2025. URL: <https://arxiv.org/abs/2506.01953>.
-

-
- [25] DAM-VLA: Decoupled Asynchronous Multimodal Vision Language Action model. arXiv preprint arXiv:2606.12105, 2026. URL: <https://arxiv.org/abs/2606.12105>.
 - [26] Y. Du et al. Learning Universal Policies via Text-Guided Video Generation. arXiv preprint arXiv:2302.00111, 2023. URL: <https://arxiv.org/abs/2302.00111>.
 - [27] WorldVLA: Towards Autoregressive Action World Model. arXiv preprint arXiv:2506.21539, 2025. URL: <https://arxiv.org/abs/2506.21539>.
 - [28] World Action Models are Zero-shot Policies. arXiv preprint arXiv:2602.15922, 2026. URL: <https://arxiv.org/abs/2602.15922>.
 - [29] Fast-WAM: Do World Action Models Need Test-time Future Imagination? arXiv preprint arXiv:2603.16666, 2026. URL: <https://arxiv.org/abs/2603.16666>.
 - [30] Cosmos Policy: Fine-Tuning Video Models for Visuomotor Control and Planning. arXiv preprint arXiv:2601.16163, 2026. URL: <https://arxiv.org/abs/2601.16163>.
 - [31] Unified Video Action Model. arXiv preprint arXiv:2503.00200, 2025. URL: <https://arxiv.org/abs/2503.00200>.
 - [32] LaWAM: Latent World Action Models for Efficient Dynamics-Aware Robot Policies. arXiv preprint arXiv:2606.15768, 2026. URL: <https://arxiv.org/abs/2606.15768>.
 - [33] Being-H0.7: A Latent World-Action Model from Egocentric Videos. arXiv preprint arXiv:2605.00078, 2026. URL: <https://arxiv.org/abs/2605.00078>.
 - [34] Hao Wu, Xuejin Tian, Minghao Li, Yunxin Liu, Ganesh Ananthanarayanan, Fengyuan Xu, and Sheng Zhong. Pecam: Privacy-enhanced video streaming and analytics via securely-reversible transformation. In *Proceedings of the 27th Annual International Conference on Mobile Computing and Networking*, pages 229–241, 2021.
 - [35] Hao Wu, Jinghao Feng, Xuejin Tian, Edward Sun, Yunxin Liu, Bo Dong, Fengyuan Xu, and Sheng Zhong. Emo: Real-time emotion recognition from single-eye images for resource-constrained eyewear devices. In *Proceedings of the 18th International Conference on Mobile Systems, Applications, and Services*, pages 448–461, 2020.
 - [36] Fei Zeng, Feng Lyu, Hao Wu, Zhanxi Li, Shucheng Li, Fengyuan Xu, and Yaoxue Zhang. H2o: Heterogeneity-aware hierarchical orchestration for memory-efficient on-device llm inference. *IEEE Transactions on Mobile Computing*, 2025.
 - [37] Borui Li, Tianen Liu, Weilong Wang, Chengqing Zhao, and Shuai Wang. Agent-as-a-service: An ai-native edge computing framework for 6g networks. *IEEE Network*, 39(2):44–51, 2024.
 - [38] Borui Li, Tiange Xia, and Shuai Wang. Infscaler: Enabling efficient ml inference serving on multi-accelerator edge devices via asymmetric auto-scaling. In *2025 62nd ACM/IEEE Design Automation Conference (DAC)*, pages 1–7. IEEE, 2025.
 - [39] Borui Li, Yitao Wang, Haoran Ma, Ligeng Chen, Jun Xiao, and Shuai Wang. Mobilora: Accelerating lora-based llm inference on mobile devices via context-aware kv cache optimization. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 23400–23410, 2025.
-

-
- [40] K. D. Nguyen, H. T. Ho, C. T. Nguyen, T. Q. Duong, L. D. Le, D. M. H. Nguyen, V. A. Ngo, and A. T. Le. vla.cpp: A Unified Inference Runtime for Vision-Language-Action Models. *arXiv preprint arXiv:2606.08094*, 2026. Submitted June 6, 2026. URL: <https://arxiv.org/abs/2606.08094>.
 - [41] I. Gim, Z. Ma, S.-S. Lee, and L. Zhong. Pie: A Programmable Serving System for Emerging LLM Applications. In *Proceedings of the 31st ACM Symposium on Operating Systems Principles (SOSP)*, 2025. URL: <https://doi.org/10.1145/3731569.3764814>.
 - [42] L. Su. Execution-State Capsules: Graph-Bound Execution-State Checkpoint and Restore for Low-Latency, Small-Batch, On-Device Physical-AI Serving. *arXiv preprint arXiv:2606.20537*, 2026. URL: <https://arxiv.org/abs/2606.20537>.